

TAPLite Kernel Computational-Efficiency Enhancements: Warm Starts, SP-Tree Sharing, and Column Stores

Xuesong Zhou

TAPLite4MPO team

July 2026

Abstract

This memo summarizes the computational-efficiency track for the TAPLite static-assignment kernel: a five-persona industry panel study that produced a four-level warm-start ladder (L1–L4); the shipped P0 trio (congested-times warm start, shortest-path tree sharing, zero-demand-origin skip); the shipped P1 flow restart with a demand-fingerprint guard; and the shipped P2/L3 binary column store with θ -share warm start and fixed-policy gradient-projection (GP) sweeps. All shipped features are bit-exact or exact-with-clearly-reported deviation, default-off where they change behavior, and gated by a 13-network / 30-check regression suite passed with no re-baselining. Headline, SCAG RTP24 AM (246,806 links, perturbed demand, same-gap compute): the L3 column warm start reaches a 0.10% TRUE relative gap **139x** faster than a cold run and 0.05% **8.4x** faster; targets at or below 0.02% are reachable *only* via the GP sweeps — plain FW, cold or warm-started, plateaus near 0.03% while GP reaches 8×10^{-4} %. The standard rerun recipe is therefore an L3 warm start with a 0.1% gap target.

1 Headline numbers for the 4-step feedback loop

SCAG’s operational question is not what a single assignment costs, but what kernel efficiency buys the *looped* 4-step process, where assignment runs once per global feedback iteration — typically 4–8 loops $\times$ 5 time-of-day periods, i.e. 20–40 assignment calls per model run. Table 1 collects the headline factors. Every entry derives only from our measured numbers (the measurement each derives from is cited); the whole-run row is arithmetic on those measurements and is labeled as such.

Table 1: Headline efficiency factors for the looped 4-step process. “Measured” entries cite the source measurement; “derived” rows are arithmetic on those measurements.

Lever	Headline factor	Basis
1. Within-loop warm start, L1 times (shipped)	~45% fewer iterations to the cold 20-iteration gap (~ 11 vs 20); 1.9x tighter gap at equal iterations	Measured: Table 2, SCAG RTP24 AM
2. Loop-over-loop restart, L2 flows (shipped)	2.2x per post-loop-1 assignment call (35 s vs 78 s); restart begins <i>at</i> the prior equilibrium gap (0.072%)	Measured: Table 3, SCAG RTP24 AM
3. Whole feedback run	~1.8x from warm starts alone	Derived from rows 1–2 (arithmetic below)
4. Scenario re-runs / demand edits, L3 columns (shipped)	139x to a 0.10% TRUE gap; 8.4x to 0.05%; 0.02%/0.01% reachable <i>only</i> via GP sweeps	Measured: Table 5, SCAG RTP24 AM perturbed demand, compute-only

Whole-run arithmetic (derived, not separately measured). A 5-loop, single-period feedback run costs $5T$ cold. With the shipped warm starts, the first assignment call costs T and each of the four subsequent calls $\sim 0.45T$ (the measured L2 wall-time ratio, Table 3): $T + 4 \times 0.45T \approx 2.8T$, i.e. **~1.8x** for the full feedback run from warm starts alone. A 5-period daily loop (25 assignment calls) scales the same way, because every call benefits.

Multi-class runs additionally saved 2–3x on the shortest-path phase via automatic tree sharing (6 modes $\rightarrow$ 4 trees, byte-identical outputs; Section 3), and every number above sits on the 30/30 regression suite plus the byte-identity gate (Section 7).

The kernel’s shortest-path engine was modernized to industry-standard heap label-setting (21x on our prior internal baseline) — treated here as a prerequisite, not a claimed advance.

2 Motivation and the industry-panel method

Regional static assignment runs are rarely one-off: agencies rerun the same network under VDF recalibration, toll tweaks, demand scenarios, and gap-tightening continuations. A cold Frank–Wolfe (FW) start wastes its first iterations rediscovering congestion the previous run already computed — on SCAG RTP24 AM the free-flow iteration-1 relative gap is 185%. Rather than guessing at accelerations, we convened a structured panel of five expert personas, each grounded in a distinct engineering tradition:

1. a **Caliper-class commercial vendor** engineer,
2. a **PTV/Visum-class commercial vendor** engineer,
3. an **open-source lead** (AequilibraE-class),
4. a **Bar-Gera-tradition academic** (algorithmic convergence theory),
5. an **HPC/systems engineer** (memory layout, binary formats, mmap).

Each persona reviewed the actual kernel source (`kernel/src/TAPLite.cpp`) and the feature inventory, proposed accelerations with effort estimates, and registered dissents. The panel converged on a single organizing artifact: a *warm-start ladder* with four rungs, each persisting a richer object from a prior run.

- L1 — Congested TIMES preload.** Seed link travel times from a prior run, so iteration-0 all-or-nothing routes on congested rather than free-flow times. Feasibility untouched; equilibrium unchanged. All five personas endorsed shipping this first.
- L2 — FLOWS restart.** Restore link volumes from a binary snapshot and skip the iteration-0 all-or-nothing entirely. Valid only when demand is identical — enforced by a demand fingerprint.
- L3 — Binary COLUMN store.** Persist per-OD path sets with θ shares. Unanimously “the strategic asset”: θ over a path set is a demand-*invariant* routing policy, replayable and rescalable against any new OD table — which neither times nor raw flows can promise.
- L4 — Full binary state snapshot (DTST).** An mmap-able section-table snapshot of network, demand, volumes, times, and columns that eliminates the input-parsing phase. Planned; valuable for Python-driven sweeps.

The panel’s universal rule: reset conjugate/bi-conjugate FW direction history on any warm start — conjugate directions from another problem are poison. Startup resolution order: valid DTAC $\rightarrow$ L3; else valid DTLR $\rightarrow$ L2; else valid DTLP $\rightarrow$ L1; else free-flow — every decision and id-match rate logged.

3 P0: the exact trio (shipped)

All three P0 features are *exact*: outputs are bit-identical with the features off (L1) or on (tree sharing and origin skip, which only remove redundant work). Exactness was verified by byte-diffing `link_performance.csv` and the gap trajectory of the new executable against the pre-change executable.

3.1 L1 warm start from congested times (`warm_start_times`)

Setting `warm_start_times` to a prior run’s `link_performance` CSV (compact or full) or binary DTLP file preloads each link’s travel time (and recomposed generalized cost) *by external link id, never row order*, before the first shortest-path pass. Only the starting point changes; later iterations recompute costs from volumes, so the equilibrium is unchanged. A missing/unreadable file or zero id matches produces a loud warning and a cold start — the run never fails. On SCAG, 224,288 of 246,806 links matched; connectors carry a zero time in the source file and correctly keep free-flow time.

Table 2: L1 warm start on SCAG RTP24 AM (224,288 links, plain FW, 20 iterations).

	Cold start	L1 warm start
Iteration-1 relative gap	185.2%	44.9%
Iterations to cold’s final gap	19	~11
Final gap (iteration 19)	0.0755%	0.0407%
Relative improvement	—	~1.9x tighter

3.2 SP-equivalence auto-detect (tree sharing)

Two user classes need separate shortest-path trees *if and only if* their `allowed_use` masks or per-link additional-cost vectors (tolls plus distance operating cost, monetized through VOT) differ on any link. PCE and occupancy never force a dedicated tree — they enter loading and gap weighting only. After input read, the kernel groups modes by exact comparison of both vectors; non-representative modes reuse the representative’s tree. On the Chicago Sketch multimodal fixture, 6 modes collapse to 4 trees with byte-identical outputs, and the grouping is always logged (e.g. “{sov,com} -> sov; {hov2,hov3} -> hov2; {trk}; {apv}”) — the panel was unanimous that silent cleverness loses MPO trust.

3.3 Group-aware zero-demand-origin skip

The kernel already skipped origins with zero *total* demand. This was completed to per-(mode, origin): the shortest-path solve is skipped when no mode in the SP group has demand out of the origin, and the route-cost row is BIGM-filled directly — exactly what the full destination loop produces for a zero demand row, so results are identical. Active/total pair counts are logged.

Regression: all 30 checks on the 13-network suite pass post-change, with no re-baselining, and the SCAG cold gap trajectory matches the pre-change run digit-for-digit.

4 P1: DTLR flow snapshot and warm_start_flows (shipped)

The L2 rung persists link flows. `flow_snapshot=1` (auto-written when `link_output=3`) writes `link_flows.bin`: magic ‘DTLR’, version, link and mode counts, a **16-byte demand fingerprint** (four floats: OD total, positive-cell count, min/max zone id touched, accumulated in deterministic fixed order), then per-link main and per-mode volumes keyed by external link id.

`warm_start_flows` restores those volumes, recomputes times from them, skips the iteration-0 all-or-nothing, and continues FW from iteration 1. The restored point is a convex combination of all-or-nothing assignments of the *same* demand, hence a valid FW iterate — this is precisely the condition the open-source panelist demanded (see Section 7).

The guard. On fingerprint mismatch, mode-count mismatch, or an unreadable/wrong-magic file, the kernel warns loudly (both fingerprints printed) and *demotes to L1*: the snapshot seeds congested times only, followed by a normal cold all-or-nothing — never a silent restart from incompatible flows. The guard was verified on Chicago Sketch: perturbing one OD cell by 10% flips the fingerprint and triggers the demotion, after which the run proceeds as a times-only warm start (iteration-1 gap 2.78% vs 510.78% cold).

Table 3: L2 flow restart on SCAG RTP24 AM (246,806 links, plain FW).

	Run A: cold, 20 iters	Run B: <code>warm_start_flows</code> , 10 iters
Links restored	—	246,806 / 246,806
Iteration-1 gap	185.23%	0.0724% (= A’s final gap)
Final gap	0.0755%	0.0453%
FW wall time	78 s	35 s
Relative improvement	—	1.7x tighter in 0.45x time

The restart begins *at* the prior run’s final gap — the continuation is seamless, making gap-tightening reruns and identical-demand recalibration loops essentially free of rediscovery cost.

5 P2: the DTAC column store (increments 1 and 2 shipped)

5.1 Increment 1: write + verify (shipped)

`column_output=1` writes `route_columns.bin`: a sparse-CSR binary (magic ‘DTAC’) storing, for every (mode, origin, destination) with positive demand and a feasible route, the *last-iteration shortest path* as an external-link-id sequence — back-traced exactly like the route CSV writer, with the same tree sharing and turn-restriction link-state walk. Increment 1 is route-assignment *parity* only: one final path per OD, not yet the multi-path column set with θ shares.

A stdlib-only verifier (`python -m dtalite.qa columns`) pushes each OD volume (PCE-weighted) down its stored path and compares per-link sums with the run’s link-performance volumes.

Table 4: DTAC increment 1 on Chicago Sketch (20 iterations, 3 modes, 387 zones).

Stored paths / link entries	93,135 / 1,211,261
DTAC file size	5.6 MB
<code>route_assignment.csv</code> size	55.1 MB
Compression	9.8x smaller
Push-down R^2 vs. link volumes	0.9657
Turn-restriction fixture (single-path net)	$R^2 = 1.0$; banned movement avoided

Honest caveat: $R^2 = 0.966$ is *expected, not a defect*. The link-performance volumes are the FW blend of all 20 iterations, while increment 1 stores only the final all-or-nothing direction; the R^2 quantifies exactly that distance. Exactness arrives with increment 2’s θ shares.

5.2 Increment 2 / full L3: θ -share columns, replay, and GP sweeps (shipped)

The full L3 scope shipped and was benchmarked: `column_output=1` now writes DTAC **v2** — the path *set* accumulated across all FW iterations with per-path θ shares (the exact FW cascade), so the push-down against link volumes is exact by construction ($R^2 = 1.000000$, max diff 0.00 veh on Chicago Sketch; the increment-1 figure was 0.9657). `warm_start_columns` loads a DTAC file and scatters $\theta \times$ the *current* OD table to link volumes — demand *may* differ from the stored fingerprint; that is the point of a demand-invariant routing policy — dropping paths broken by network edits (counted, warned, θ renormalized), then skips the iteration-0 all-or-nothing and resumes FW. `column_adjust_sweeps=N` runs N fixed-policy gradient-projection sweeps over the stored columns (Gauss–Seidel, exact VDF derivatives, no shortest-path calls). After any column warm start the kernel *always* prints both a clearly-labeled **restricted gap** (stored paths only) and the **TRUE relative gap** from fresh shortest paths — a restricted equilibrium never masquerades as UE. Freeze/replay (`number_of_iterations=0` + columns) is pure policy replay with the same mandatory gap report.

5.3 The same-gap trade-off study

The operative question is *compute seconds to reach a given TRUE-gap target* on a rerun. Setup (network + demand parse + DTAC load) is excluded by an explicit accounting rule — it is common to every alternative and is the target of the planned L4 state snapshot; the kernel prints phase timings so users can apply the same rule.

The regime split is the finding of Figure 1: plain FW — cold *or* warm-started — plateaus above $\sim 0.03\%$ on these networks, while GP sweeps over the stored columns pass 0.01% in 1–3 sweeps and keep going (SCAG, 8 sweeps: $7.6 \times 10^{-4}\%$). The sweeps are not merely faster to a fixed target; they reach targets FW cannot. Their cost scales with OD count (SCAG 343k ODs: ~ 23 s first sweep, ~ 11 s after; Chicago Regional 2.3M ODs: ~ 60 s), which motivates the remaining per-origin parallelization work.

Table 5: Same-gap trade-off, SCAG RTP24 AM (246,806 links, 342,712 OD cells; demand perturbed +5% on 1,000 cells). Compute seconds to reach a TRUE-gap target; cold FW run to 80 iterations, warm replay+FW to 40, GP points = final TRUE gap of separate `column_adjust_sweeps`= k replay runs.

Target	Cold FW	Warm replay + FW	Warm + GP sweeps	Speedup
0.10%	101.7 s @ 15 it	8.6 s @ 1 it	0.7 s @ scatter	139x
0.05%	183.4 s @ 26 it	28.5 s @ 6 it	21.9 s @ 1 sweep	8.4x
0.02%	not reached (80 it → 0.036%)	not reached (40 it → 0.033%)	21.9 s @ 1 sweep	only GP
0.01%	not reached	not reached	21.9 s @ 1 sweep	only GP

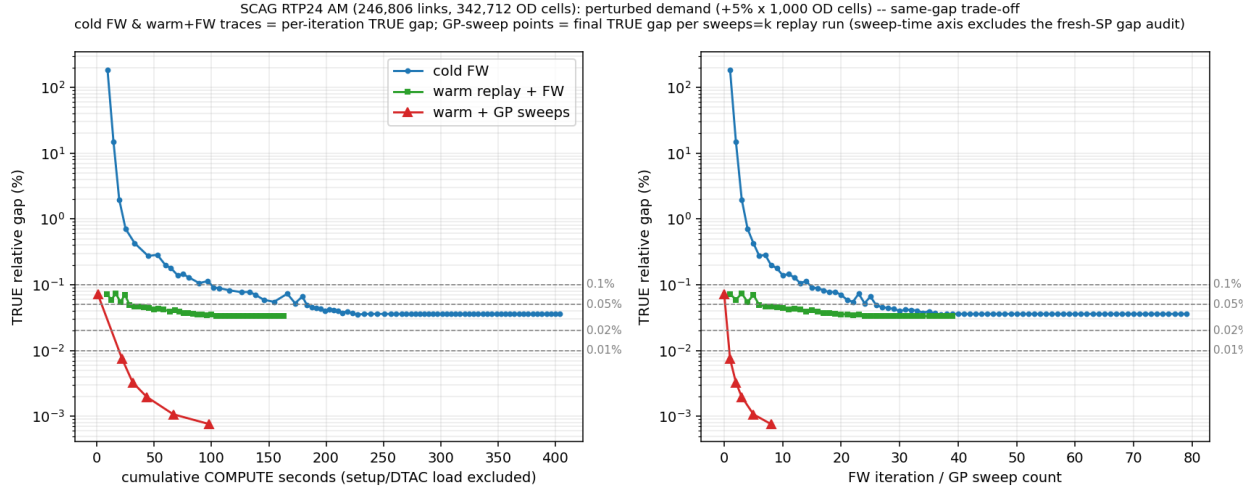


Figure 1: SCAG RTP24 AM same-gap trade-off. Left: TRUE relative gap vs cumulative compute seconds (setup/DTAC load excluded); right: vs FW iteration / GP sweep count. FW curves are per-iteration TRUE gap; GP points are the final TRUE gap of each sweeps= k run. Companion figure `l3_tradeoff_sketch.png` shows the same regimes on Chicago Sketch (0.10% → 4.8x, 0.05% → 2.4x, 0.02%/0.01% → GP only).

The standard rerun recipe (team decision). Rerunning *any* model = L3 column warm start with a 0.1% relative-gap target:

```
warm_start_columns = route_columns.bin  convergence_gap_pct = 0.1
column_adjust_sweeps = 0
```

(the baseline run sets `column_output=1` once). Replay+FW reaches the 0.1% TRUE gap in ~1 iteration — on SCAG, 139x less compute than a cold run to the same gap. Add GP sweeps only for tighter targets (1 sweep reaches 0.01% on SCAG). Kernel cold-start defaults are unchanged; the recipe is settings-only. Full recipe and guard behavior: `github:taplite/docs/RERUN_RECIPE.md`.

6 The binary format family

Table 6: TAPLite binary formats. All carry magic + version headers, match by external link id, and fall back loudly (never fail) on invalid input.

Magic	Role	Written by	Consumed by
DTAB	OD demand input	<code>dtalite_qa demand-bin</code>	<code>demand.format=1</code> (2.5–3x faster load)
DTLP	Link performance	<code>link_output=3</code>	L1 <code>warm_start_times</code> ; reporting
DTLR	Flow restart snapshot	<code>flow_snapshot=1</code>	L2 <code>warm_start_flows</code>
DTAC	Path/column store (v2: θ shares)	<code>column_output=1</code>	verifier; L3 <code>warm_start_columns</code>
DTST	Full state snapshot	<code>planned</code> (L4)	<code>planned</code> : mmap, skips input parse

7 Engineering principles

The efficiency track is governed by rules that came directly out of the panel and are enforced on every increment:

- **Exact or clearly reported.** A feature is either bit-exact (P0 trio, L2 under a matching fingerprint) or its deviation is quantified and printed (DTAC increment-1 R^2 ; the future restricted-vs-full gap split).
- **Default-off.** Every behavior-changing key (`warm_start_times`, `warm_start_flows`, `flow_snapshot`, `column_output`) defaults off and is registered in the `dtalite_qa` schema; existing runs are byte-identical.
- **Loud-warn, never fail.** Bad warm-start input — missing file, wrong magic, zero id matches, unmatched links from network edits — warns loudly and degrades gracefully (cold start or L1 demotion); a production run is never aborted by an optimization hint.
- **Fingerprint guards.** Flow restarts are gated by an exact demand fingerprint; a single perturbed OD cell flips it. Grouping and warm-start decisions are always logged with match rates.
- **Regression + byte-identity gates.** Every increment must pass the 13-network / 30-check suite with *no re-baselining*, plus byte-diffs of `link_performance.csv` and gap trajectories against the pre-change executable.
- **No bush-based rewrite.** The panel unanimously rejected an Algorithm-B/TAPAS rewrite. The academic persona’s framing: gradient- projection polish on stored columns delivers “bush-method-class precision without becoming Algorithm B” — disproportionate machinery for a single-file link-based FW kernel is exactly what this track avoids.

Summary

P0, P1, and the full P2/L3 are shipped and verified: warm starts turn SCAG reruns from a 185%-gap cold open into a continuation that starts at the prior final gap, and the L3 column store turns *any* rerun — including changed demand — into a 0.1%-gap solution in ~ 1 FW iteration (139x less compute than cold on SCAG), with GP sweeps unlocking gap targets FW cannot reach at all. The standard rerun recipe is now fixed: `warm_start_columns + convergence_gap_pct=0.1` + zero sweeps, with zero change to any run that does not opt in (30/30 regression, defaults-off byte-identity). Remaining work: per-origin parallel GP sweeps (the serial sweeps dominate at ≥ 2 M ODs) and the L4 DTST state snapshot to kill the setup phase. The ladder compounds: each rung reuses the formats and guards of the one below it.